```
import pandas as pd
import numpy as np

df = pd.read_csv("retail_relabel.csv")
print(df.shape)
print(df.columns.tolist())
print(df.head())
```

```
(1500, 13)
['SKU', 'Category', 'UnitCost', 'StorePrice', 'CostChangePct', 'Competitor_BigBoxDepot', 'Competitor_ValueMart', 'Competitor_Qui
        SKU     Category  UnitCost  StorePrice  CostChangePct  \
0  SKU10000      Grocery      1.74        3.06           6.98
1  SKU10001  Electronics     31.67       54.91           5.86
2  SKU10002    Household     17.18       24.37           2.27
3  SKU10003    Household     12.08       15.76          -8.49
4  SKU10004    Household      4.75        6.97           7.41

   Competitor_BigBoxDepot  Competitor_ValueMart  Competitor_QuickShop  \
0                    2.46                  3.34                  2.22
1                   42.03                 52.81                 40.01
2                   26.55                 28.43                 26.72
3                   15.80                 15.93                 18.13
4                    6.57                  6.96                  6.41

   WeeklyUnitsSold  DaysSinceLastChange  ElectronicShelfLabel  RelabelCost  \
0               71                  108                     0         0.47
1               34                   51                     0         0.45
2               91                  196                     1         0.06
3               54                  254                     1         0.05
4               52                  121                     0         1.75

   Relabel
0        1
1        0
2        1
3        1
4        0
```

```
df['CompAvg'] = df[['Competitor_BigBoxDepot', 'Competitor_ValueMart', 'Competitor_QuickShop']].mean(axis=1)
df['GapPct'] = 100 * (df['StorePrice'] - df['CompAvg']) / df['CompAvg']
df['AbsGapPct'] = df['GapPct'].abs()
df['LogUnits'] = np.log(df['WeeklyUnitsSold'])
df['MarginPct'] = 100 * (df['StorePrice'] - df['UnitCost']) / df['StorePrice']

print(df.head())
```

```
        SKU     Category  UnitCost  StorePrice  CostChangePct  \
0  SKU10000      Grocery      1.74        3.06           6.98
1  SKU10001  Electronics     31.67       54.91           5.86
2  SKU10002    Household     17.18       24.37           2.27
3  SKU10003    Household     12.08       15.76          -8.49
4  SKU10004    Household      4.75        6.97           7.41

   Competitor_BigBoxDepot  Competitor_ValueMart  Competitor_QuickShop  \
0                    2.46                  3.34                  2.22
1                   42.03                 52.81                 40.01
2                   26.55                 28.43                 26.72
3                   15.80                 15.93                 18.13
4                    6.57                  6.96                  6.41

   WeeklyUnitsSold  DaysSinceLastChange  ElectronicShelfLabel  RelabelCost  \
0               71                  108                     0         0.47
1               34                   51                     0         0.45
2               91                  196                     1         0.06
3               54                  254                     1         0.05
4               52                  121                     0         1.75

   Relabel    CompAvg     GapPct  AbsGapPct  LogUnits  MarginPct
0        1   2.673333  14.463840  14.463840  4.262680  43.137255
1        0  44.950000  22.157953  22.157953  3.526361  42.323803
2        1  27.233333 -10.514076  10.514076  4.510860  29.503488
3        1  16.620000  -5.174489   5.174489  3.988984  23.350254
4        0   6.646667   4.864594   4.864594  3.951244  31.850789
```

```
print("Overall relabel rate:", df['Relabel'].mean().round(3))

print("\nBy Category:")
print(df.groupby('Category')['Relabel'].mean().round(3))
```

```
print("\nBy ESL:")
print(df.groupby('ElectronicShelfLabel')['Relabel'].agg(['size', 'mean']).round(3))
```

```
Overall relabel rate: 0.4

By Category:
Category
Apparel         0.390
Electronics     0.440
Grocery         0.412
Household       0.378
PersonalCare    0.383
Name: Relabel, dtype: float64

By ESL:
                      size   mean
ElectronicShelfLabel
0                     1045  0.339
1                      455  0.541
```

Interpretation: Overall, 40% of tags got changed, and the rate is pretty similar across categories (38–44%). The big difference is the label type: items with electronic shelf labels got repriced 54.1% of the time, versus only 33.9% for paper tags. This is the menu cost in action, when fixing a price costs 5 cents, small mistakes get fixed; when it costs $1–2 in labor and materials, about two thirds of wrong tags just stay wrong.

```
import statsmodels.formula.api as smf

model1 = smf.logit('Relabel ~ GapPct + UnitCost + RelabelCost', data=df).fit()
print(model1.summary2().tables[1].round(4))
print("Pseudo R2:", round(model1.prsquared, 4), "| AIC:", round(model1.aic, 1))
```

```
Optimization terminated successfully.
         Current function value: 0.621007
         Iterations 5
              Coef.  Std.Err.       z   P>|z|  [0.025  0.975]
Intercept    0.0589    0.0977  0.6028  0.5466 -0.1326  0.2505
GapPct       0.0379    0.0045  8.3831  0.0000  0.0290  0.0467
UnitCost     0.0005    0.0032  0.1545  0.8772 -0.0058  0.0068
RelabelCost -0.6387    0.0707 -9.0315  0.0000 -0.7773 -0.5001
Pseudo R2: 0.0773 | AIC: 1871.0
```

Interpretation: GapPct comes out statistically significant ($p < 0.001$), but the model overall is weak: pseudo R^2 is only 0.077 and AIC is 1871. The problem is that the signed gap asks the wrong question. The owner reprices an item because its price is wrong, not because it's wrong in a particular direction. 10% over competitors loses customers, 10% under gives away margin and both get fixed. But the signed version records those as +10 and −10, so items that both got relabeled point the model in opposite directions and mostly cancel each other out. The real effect gets watered down to a small coefficient (0.038). And one lesson from this: a significant p-value doesn't mean a strong effect with 1,500 rows, even a watered-down effect passes the significance test.

```
model2 = smf.logit('Relabel ~ AbsGapPct + CostChangePct + LogUnits + RelabelCost + DaysSinceLastChange + UnitCost', data=df).fi
print(model2.summary().tables[1])
print("Pseudo R2:", round(model2.prsquared, 4), "| AIC:", round(model2.aic, 1))
```

```
Optimization terminated successfully.
         Current function value: 0.487939
         Iterations 6
==============================================================================================
                         coef    std err          z      P>|z|      [0.025      0.975]
----------------------------------------------------------------------------------------------
Intercept              -4.2772      0.567     -7.540      0.000      -5.389      -3.165
AbsGapPct               0.1778      0.011     16.245      0.000       0.156       0.199
CostChangePct           0.1276      0.017      7.584      0.000       0.095       0.161
LogUnits                0.4324      0.131      3.300      0.001       0.176       0.689
RelabelCost            -0.8589      0.085    -10.085      0.000      -1.026      -0.692
DaysSinceLastChange     0.0040      0.001      6.369      0.000       0.003       0.005
UnitCost                0.0004      0.004      0.102      0.919      -0.007       0.008
==============================================================================================
Pseudo R2: 0.275 | AIC: 1477.8
```

```
import pandas as pd

odds = pd.DataFrame({
    'OddsRatio': np.exp(model2.params),
    'CI_low': np.exp(model2.conf_int()[0]),
    'CI_high': np.exp(model2.conf_int()[1])
```

```
}).round(3)
print(odds)
```

```
                     OddsRatio  CI_low  CI_high
Intercept                0.014   0.005    0.042
AbsGapPct                1.195   1.169    1.220
CostChangePct            1.136   1.099    1.174
LogUnits                 1.541   1.192    1.992
RelabelCost              0.424   0.359    0.501
DaysSinceLastChange      1.004   1.003    1.005
UnitCost                 1.000   0.993    1.008
```

Interpretation: Switching from the signed gap to the absolute gap, and adding the features the business reasoning called for, improves everything: pseudo R^2 jumps from 0.077 to 0.275, AIC drops from 1871 to 1477.8 (BIC shows the same), and the gap coefficient nearly quintuples (0.038 → 0.178) the effect was there all along, the signed version was just hiding it. Every extra percentage point of price gap versus competitors raises the odds of a relabel by about 20% (OR 1.195, CI 1.169–1.220). Every extra dollar it costs to change a tag cuts the odds of the change happening by more than half (OR 0.424, CI 0.359–0.501) the menu cost directly discouraging fixes.Fast sellers get repriced more readily (LogUnits OR 1.541), which makes sense: a tag change pays for itself in days on an item selling 60 units a week but takes months on a slow mover. Finally, the model confirms what economic reasoning predicts: a cost increase since the last labeling strongly predicts a relabel (CostChangePct OR 1.136, $p < 0.001$), but how expensive an item is doesn't matter at all (UnitCost OR 1.000, $p = 0.919$). That's because the current price was set knowing the item's cost, the cost level is already baked into the tag. Only a cost change makes the tag wrong. Model 2 is the one to deploy.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, roc_auc_score

train, test = train_test_split(df, test_size=0.3, stratify=df['Relabel'], random_state=7)

model3 = smf.logit('Relabel ~ AbsGapPct + CostChangePct + LogUnits + RelabelCost + DaysSinceLastChange + UnitCost', data=train)

test_probs = model3.predict(test)          # each test SKU's x-Relabel probability
test_preds = (test_probs >= 0.5).astype(int)   # 0.5 threshold → yes/no calls

print(confusion_matrix(test['Relabel'], test_preds))
print("Accuracy:", round(accuracy_score(test['Relabel'], test_preds), 3))
print("AUC:", round(roc_auc_score(test['Relabel'], test_probs), 3))
```

```
Optimization terminated successfully.
         Current function value: 0.501281
         Iterations 6
[[233  37]
 [ 64 116]]
Accuracy: 0.776
AUC: 0.853
```

Interpretation: Everything up to now graded the model on the same data it learned from,like grading a student on the practice exam they studied. Part 4 is the real exam: the model trained on 70% of the SKUs and predicted the 30% it had never seen. It got 77.6% of those right, which matters because just predicting "no relabel" for everything would already score about 60%, so the model genuinely adds about 18 points over doing nothing. The AUC of 0.853 means that if you hand it one relabeled item and one untouched item, it ranks the relabeled one as more likely 85% of the time. Its mistakes lean one way: it misses actual relabels (64) more than it raises false alarms (37) it's more likely to leave a wrong tag alone than to send the crew to a fine one. The small drop from training performance tells us the model actually learned the pattern rather than memorizing the data, with only six features, all chosen for business reasons, there wasn't much room to memorize anyway.

```
test = test.copy()   # avoids a pandas warning about writing to a slice

test['ExpectedWeeklyLoss'] = (test['AbsGapPct']/100) * test['StorePrice'] * test['WeeklyUnitsSold']
test['NetBenefit'] = test['ExpectedWeeklyLoss'] - test['RelabelCost']

share = (test['NetBenefit'] > 0).mean()
print("Share recouping relabel cost within one week:", round(share, 3))
print(test[['ExpectedWeeklyLoss','NetBenefit']].describe().round(2))
```

```
Share recouping relabel cost within one week: 0.987
       ExpectedWeeklyLoss  NetBenefit
count              450.00      450.00
mean               135.94      134.96
std                342.13      342.19
min                  0.21       -2.07
25%                 17.44       16.49
```

```
50%              43.18      42.26
75%             135.35     133.48
max            5293.25    5292.82
```

Interpretation:

This is where the statistics turn into money. For each test SKU we estimated the weekly cost of leaving its wrong price alone, and compared it against the cost of fixing the tag. The result: 98.7% of fixes would pay for themselves within a single week. The typical mispriced item leaks about $43 a week against a fix that costs between 5 cents and 2 dollars, and the worst one leaks an estimated $5,293 a week behind a 43-cent tag change. These loss numbers are on the high side by construction, the formula assumes the full price gap is lost every week, which won't literally be true but even if you cut them substantially, the conclusion doesn't change: what's holding back pricing accuracy isn't knowing which tags are wrong, it's the cost of changing them.

MEMO — Repricing Policy Recommendations

To: Store Owner, SunCoast Retail Mart

From: Gabriel Gutierrez

Re: Logistic model of the relabeling decision, one pricing cycle (1,500 SKUs)

(a) What to relabel first. Priority should go to high-velocity SKUs with large price misalignment, in our data, concentrated in Electronics, where competitor prices move fastest and gaps blow out quickly. The case is stark: our single largest leak is an Electronics SKU losing an estimated $5,293 per week to mispricing, behind a $0.43 tag fix; other top-ten fixes cost as little as $0.05–$0.07. These corrections are near-certain to pay off — 98.7% of test SKUs would recoup their relabel cost within one week of corrected pricing.

(b) The case for converting more shelves to electronic labels. Paper tags don't just cost labor, they stop corrections from happening: paper-tag items were repriced at 34% versus 54% for ESL items, meaning roughly two-thirds of wrong paper tags stayed wrong, each leaking a median of about $43 per week in either margin left on the table or customers lost to lower-priced competitors. Every shelf converted turns a one-to-two-dollar decision into a five-cent decision, and our model estimates each dollar of relabel friction cuts the odds of a correction by roughly 58%. Even discounting our loss estimates heavily they assume the full price gap is lost weekly, an upper bound the conversion pays for itself quickly in recovered pricing accuracy.

(c) A caution on causality. These data are observational: nobody randomly assigned electronic labels to shelves, and the converted shelves were likely chosen precisely because they were volatile, high-benefit sections. ESL is therefore *associated* with more repricing; we cannot yet claim it *causes* the full 20-point difference, or promise the same jump on the calmer remaining shelves — though even stable categories face shocks (a fabric shortage, a failed avocado crop) where tag friction delays the correction. To measure the true effect, convert a random sample of remaining shelves, not the most promising ones and compare their repricing behavior against unconverted shelves.

1. Model 1 vs Model 2. The signed gap treated "10% too high" and "10% too low" as opposites when the owner treats both as wrong prices needing a fix. Feeding the model opposite signs for the same behavior watered the effect down. Fixing the functional form, absolute gap instead of signed, raised pseudo R^2 from 0.077 to 0.275, dropped AIC from 1871 to 1477.8, and nearly quintupled the gap coefficient. Same data, same relationship; the encoding was hiding it.
2. MarginPct. We calculated it but never used it in a model. It probably should have been tested: an item with a thin margin has no room for pricing mistakes, the same error that barely dents a high-margin item can push a thin-margin item into selling at a loss. So low margin should make repricing more urgent, and that's information AbsGapPct doesn't capture, since it only looks at competitors, not our own costs. One caution though: MarginPct is calculated from StorePrice, and the owner is the one who sets StorePrice. So this feature is partly built out of the owner's own decisions, we'd be using the owner's choices to predict the owner's choices, which can make the model look smarter than it is. Worth adding, but with that caveat in mind.
3. In-sample vs out-of-sample. The pseudo R^2 and AIC in Parts 2–3 measure how well the model fits the data it studied — the practice exam. The accuracy and AUC in Part 4 measure how it performs on data it never saw. Good practice scores only count if the real exam confirms them, and here it did: 77.6% accuracy and 0.853 AUC on unseen SKUs.
4. From statistics to business value. Model 2 was significant and well-fit, but nothing in a regression table tells the owner what to do. The decision only appeared in Part 5, when probabilities became dollars: which tags leak the most, whether fixes pay for themselves (98.7% do within a week), and what the real bottleneck is (the cost of changing tags, not knowledge of which are wrong). And even then, the memo carries the caution that this data is observational, we can say ESL items get repriced more, not that ESL alone causes it.